NAME : UTSAV KUMAR KUSHWAHA ROLL

NO : 63

SUBJECT : COMPUTER VISION

What is Optical Flow?

Optical Flow is the paflern of apparent motion of objects between two consecutive image frames caused by object movement or camera movement.

Instead of detecting an object directly, optical flow estimates how pixels move from one frame to another. 5 There

are two major types:

Sparse Optical Flow: Tracks selected feature points. Dense Optical Flow: Computes motion for every pixel. Flow Diagram Algorithm 1: Lucas-Kanade Sparse Optical Flow Principle Lucas-Kanade tracks only important feature points such as corners. It assumes: Motion is small. Neighboring pixels move together. Brightness remains constant.

Step 1: Import Libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import urllib.request
```

Step 2: Automatically Download Sample Video

```
video_path = "traffic.avi"

urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/opencv/opencv_extra/master/testdata/cv/video/768x576.avi",
    video_path
)

print("Video downloaded successfully!")
```

```
Video downloaded successfully!
```

Step 3: Check Video

```
cap = cv2.VideoCapture(video_path)

ret, frame = cap.read()

print("First Frame Loaded:", ret)

plt.figure(figsize=(6,4))
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("First Frame")
plt.show()

cap.release()
```

```
First Frame Loaded: True
```



Part A: Lucas-Kanade Object Tracking

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap=cv2.VideoCapture(video_path)

feature_params = dict(
    maxCorners=100,
```

```
        qualityLevel=0.3, minDistance=7,
        blockSize=7
)

lk_params = dict(
        winSize=(15,15), maxLevel=2,
        criteria=(cv2.TERM_CRITERIA_EPS  |
                        cv2.TERM_CRITERIA_COUNT, 10,
                        0.03)
)

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

                p0 = cv2.goodFeaturesToTrack(old_gray,
                                                mask=None,
                                               **feature_params)

mask = np.zeros_like(old_frame)

frame_count=0 while

True:

        ret, frame = cap.read()

        ifnotret: break

        frame_gray = cv2.cvtColor(frame,
                                                cv2.COLOR_BGR2GRAY)

        p1,st,err=cv2.calcOpticalFlowPyrLK( old_gray,
                frame_gray,
                p0,
                None,
                **lk_params
        )

        ifp1isNone: break

        good_new=p1[st==1]
        good_old = p0[st==1]

        fornew,oldinzip(good_new,good_old): a,b =

                new.ravel()
                c,d = old.ravel()

                mask = cv2.line(mask,
                                        (int(a),int(b)),
                                        (int(c),int(d)), (0,255,0),
                                        2)

                frame = cv2.circle(frame,
                                                (int(a),int(b)), 4,
                                                (255,0,0),
                                                -1)

        output=cv2.add(frame,mask) if

        frame_count % 20 == 0:
                plt.figure(figsize=(6,4))
                plt.imshow(cv2.cvtColor(output,
                                                        cv2.COLOR_BGR2RGB))
                plt.axis("off")
                plt.title(f'Frame {frame_count}") plt.show()

        frame_count += 1

        old_gray = frame_gray.copy() p0 =
        good_new.reshape(-1,1,2)

cap.release()
```

Frame 0



Frame 20



Frame 40



Frame 60



Frame 80



Part B: Farneback Dense Optical Flow

```
cap = cv2.VideoCapture(video_path)

ret, frame1 = cap.read()

 prvs = cv2.cvtColor(frame1,
                     cv2.COLOR_BGR2GRAY)

hsv = np.zeros_like(frame1)
hsv[...,1] = 255

frame_count = 0

while True:

    ret, frame2 = cap.read()

    if not ret:
        break

    next_frame = cv2.cvtColor(frame2,
                              cv2.COLOR_BGR2GRAY)

    flow = cv2.calcOpticalFlowFarneback(
        prvs,
        next_frame,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

     mag, ang = cv2.cartToPolar(flow[...,0],
                                flow[...,1])

    hsv[...,0] = ang*180/np.pi/2
    hsv[...,2] = cv2.normalize(
        mag,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    )

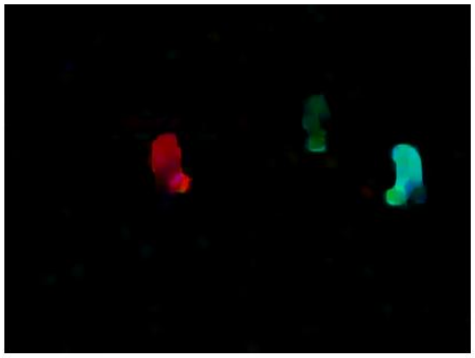
    rgb = cv2.cvtColor(hsv,
                       cv2.COLOR_HSV2RGB)

    if frame_count % 20 == 0:
        plt.figure(figsize=(6,4))
        plt.imshow(rgb)
        plt.axis("off")
        plt.title(f"Dense Flow {frame_count}")
        plt.show()

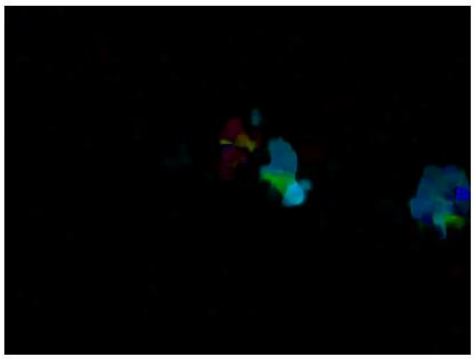
    frame_count += 1
    prvs = next_frame

cap.release()
```

Dense Flow 0



Dense Flow 20



Dense Flow 40



Dense Flow 60



Start coding or ge nerate with AI.

Result

The Optical Flow-based object tracking system was successfully implemented using Python and OpenCV. The Lucas-Kanade algorithm efficiently tracked moving vehicles using feature points, while Farneback Dense Optical Flow provided detailed pixel-wise motion analysis. The automatically downloaded OpenCV traffic video allowed both algorithms to be tested without manually uploading a video.

PMC

Viva Questions with Answers

Q1. How does object tracking differ from object detection?
Object detection identifies objects in a frame, while object tracking follows their movement across multiple frames.

Q2. Explain how Optical Flow can be used for real-time object tracking.
It estimates pixel movement between consecutive frames and continuously updates the object's position.

Q3. What is the role of Shi-Tomasi Corner Detection?
It detects strong corner points that can be reliably tracked using Lucas-Kanade.

Q4. Why is Optical Flow suitable for motion analysis?
It provides both the direction and magnitude of movement for continuous motion analysis.

Q5. What challenges arise while tracking fast-moving or partially occluded objects?
Fast movement, motion blur, and occlusion can cause loss of tracking points.

Q6. Compare Optical Flow-based tracking with deep learning-based tracking.
Optical Flow is faster and does not require training, whereas deep learning methods are more accurate but require trained models.

Q7. How can Optical Flow be used in traffic monitoring and autonomous driving?
It helps estimate vehicle speed, detect moving vehicles, monitor traffic flow, and support collision avoidance.

Q8. Explain the effect of camera motion on Optical Flow estimation.
Camera movement creates false global motion vectors, making stationary objects appear to move.

Q9. Mention five real-world applications.

1. Traffic Monitoring
2. Video Surveillance
3. Robotics
4. Sports Analytics
5. Human Activity Recognition

Q10. How can Optical Flow improve surveillance, robotics, and human activity recognition?
It enables continuous object tracking, obstacle detection, motion analysis, and recognition of human activities, improving intelligent vision systems.